



université
PARIS-SACLAY

Master 2 Innovation, Marché et Sciences des Données

RAPPORT DU PROJET DE SCALA

*Simulation du déplacement de tondeuses sur
une grille : conception orientée objet en
Scala*

Rédigé par :

KOURAOGO Emmanuel

Encadrant :

GUERNOUG Marwan

Année universitaire 2025-2026

TABLE DES MATIERES

INTRODUCTION.....	1
I. Modélisation et conception	1
II. Architecture et structuration du code	2
1. Organisation en packages.....	2
2. Séparation des responsabilités.....	4
III. Choix techniques et justification des décisions de conception	4
1. Immutabilité	4
2. Pattern matching.....	5
3. Utilisation des collections fonctionnelles.....	5
4. Gestion des erreurs	5
IV. Tests et validation.....	6
CONCLUSION	7

INTRODUCTION

Ce projet consiste à développer en Scala un algorithme simulant le déplacement de tondeuses automatiques sur une grille rectangulaire. La grille est définie par des dimensions maximales, et chaque tondeuse possède une position initiale ainsi qu'une orientation parmi les points cardinaux (Nord, Est, Sud, Ouest). À partir d'un fichier d'entrée structuré, le programme lit la configuration de la grille et les instructions associées à chaque tondeuse, puis applique ces instructions séquentiellement. Chaque instruction correspond à une rotation (gauche ou droite) ou à un déplacement vers l'avant, sous réserve du respect des limites de la grille.

Au-delà de la résolution algorithmique du problème, ce projet soulève une question centrale de conception logicielle : comment proposer une solution robuste, modulaire et conforme aux principes de la programmation orientée objet en Scala ? L'objectif n'est pas uniquement d'obtenir le résultat attendu, mais également de structurer le code de manière claire et maintenable, en appliquant des principes tels que la séparation des responsabilités, l'immuabilité des données, le pattern matching et une gestion explicite des erreurs. La démarche adoptée vise ainsi à concilier rigueur algorithmique et qualité architecturale.

I. Modélisation et conception

La première étape de conception a consisté à analyser le besoin fonctionnel afin d'identifier les entités essentielles du problème. La simulation repose sur trois éléments principaux : la position d'une tondeuse sur une grille, son orientation et son comportement de déplacement. La grille est définie par deux bornes maximales (maxX , maxY) qui délimitent l'espace autorisé. Chaque tondeuse est caractérisée par une position exprimée en coordonnées cartésiennes (x , y) et par une orientation parmi les quatre points cardinaux (N, E, S, W). Les instructions fournies dans le fichier d'entrée sont appliquées séquentiellement et correspondent à une rotation ou à un déplacement vers l'avant. Une contrainte fondamentale est le respect strict des limites de la grille : toute tentative de sortie est ignorée afin de garantir la cohérence de l'état final.

La modélisation a été réalisée selon les principes de la programmation orientée objet, en attribuant à chaque composant une responsabilité clairement définie. La classe ***Position*** représente une coordonnée cartésienne et encapsule la logique de calcul du déplacement en fonction de l'orientation. Définie comme une case class, elle bénéficie automatiquement de

méthodes utilitaires telles que *equals*, *toString* et *copy*, facilitant la manipulation d'objets immuables.

L'orientation est modélisée par un sealed trait accompagné de quatre case objects (*North*, *East*, *South*, *West*). Le mot-clé *sealed* garantit que l'ensemble des orientations possibles est connu à la compilation, permettant un pattern matching exhaustif et sécurisé. Chaque orientation encapsule sa propre logique de rotation via les méthodes *left* et *right*, ce qui centralise le comportement et évite l'usage de structures conditionnelles complexes.

La classe *Mower* regroupe l'état complet d'une tondeuse : position, orientation et limites de la grille. Elle encapsule les comportements métier (*turnLeft*, *turnRight*, *move*) et assure la validation des déplacements avant toute mise à jour. Comme *Position*, elle est définie sous forme de case class, permettant l'utilisation de la méthode *copy* pour produire une nouvelle instance modifiée.

Un choix structurant de conception est l'adoption de l'immutabilité. Aucune modification directe d'état n'est effectuée : chaque transformation retourne un nouvel objet représentant l'état mis à jour. Cette approche supprime les effets de bord, facilite le raisonnement sur le programme et renforce la robustesse de l'implémentation, tout en s'inscrivant dans une logique cohérente entre programmation orientée objet et programmation fonctionnelle.

II. Architecture et structuration du code

1. Organisation en packages

L'architecture du projet repose sur une séparation claire des responsabilités, conformément aux bonnes pratiques de conception logicielle. Le code est structuré en plusieurs packages distincts permettant de distinguer la logique métier, la logique applicative et le point d'entrée du programme.



Le package **model** regroupe les classes représentant les entités fondamentales du domaine, à savoir **Position**, **Orientation** et **Mower**. Ces classes encapsulent les données ainsi que les comportements associés à la simulation, garantissant que la logique métier demeure indépendante des aspects techniques liés à la lecture du fichier ou à l'exécution des instructions.

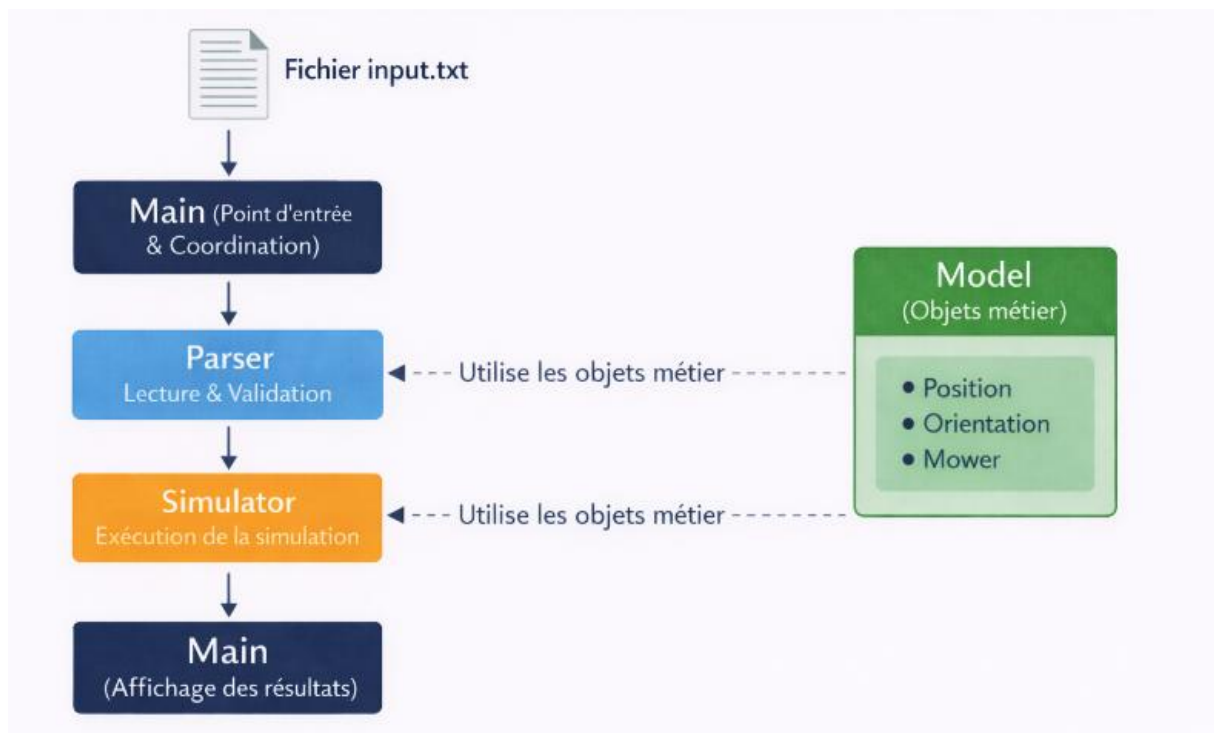
Le package **service** contient les composants responsables du traitement applicatif. Il assure la transformation des données issues du fichier d'entrée et l'application des règles de simulation, sans redéfinir la logique métier elle-même.

L'objet **Main** constitue le point d'entrée du programme. Son rôle se limite à coordonner les différentes étapes du traitement : lecture du fichier, exécution des instructions et affichage des résultats. Il ne contient aucune logique métier complexe, ce qui contribue à la clarté de l'architecture.

Cette organisation respecte le principe de responsabilité unique (Single Responsibility Principle), selon lequel chaque composant doit avoir un rôle précis et limité. Elle favorise ainsi la lisibilité, la maintenabilité et l'évolutivité du système.

L'architecture générale de l'application est illustrée dans la Figure 1, qui met en évidence la séparation entre modélisation, services et orchestration.

Figure 1 - Architecture générale de l'application



2. Séparation des responsabilités

La séparation des rôles entre les composants peut être synthétisée de la manière suivante : le *Parser* est chargé de la lecture et de la validation du fichier d'entrée ; le *Simulator* applique séquentiellement les instructions aux objets *Mower* ; le package *model* définit les entités métier et leurs comportements ; enfin, *Main* assure la coordination globale du programme.

Le *Parser* transforme les données brutes en objets métier exploitables tout en intégrant une validation minimale (format correct, orientation valide, fichier non vide). Le *Simulator* s'appuie exclusivement sur les méthodes définies dans le modèle pour exécuter les instructions, ce qui évite de mélanger logique d'exécution et représentation des données. Le modèle constitue ainsi le cœur fonctionnel du système, indépendant des mécanismes externes.

Cette architecture modulaire présente plusieurs avantages : elle améliore la lisibilité du code, facilite la maintenance et permet d'envisager des extensions futures, telles qu'une interface graphique ou une API. Elle contribue également à une meilleure testabilité, chaque composant pouvant être validé indépendamment.

III. Choix techniques et justification des décisions de conception

Au-delà de la simple implémentation algorithmique, plusieurs choix techniques ont été réalisés afin de garantir une solution robuste, maintenable et cohérente avec les paradigmes du langage Scala. Ces décisions traduisent une volonté de combiner programmation orientée objet et approche fonctionnelle, tout en respectant les bonnes pratiques de conception logicielle.

1. Immutabilité

Un principe central de l'implémentation est l'immutabilité des objets. Aucune variable mutable (*var*) n'est utilisée dans le projet. Les entités métier telles que *Position*, *Mower* et *Orientation* sont conçues comme des structures immuables, principalement définies sous forme de *case class*. Toute transformation d'état est effectuée via la méthode *copy*, qui retourne une nouvelle instance mise à jour plutôt que de modifier l'objet existant.

Ainsi, les méthodes *turnLeft*, *turnRight* et *move* produisent chacune un nouvel objet représentant l'état courant de la tondeuse. Cette approche supprime les effets de bord, améliore

la sûreté du programme et rend le comportement plus prévisible. Elle s'inscrit également dans le style fonctionnel encouragé en Scala, facilitant le raisonnement sur le code et réduisant les risques d'erreurs liés aux états partagés.

2. Pattern matching

Le *pattern matching* constitue un élément central de la solution. Il est utilisé pour gérer les orientations, interpréter les instructions de déplacement, traiter les résultats de type `Try` (`Success` / `Failure`) et valider la structure des données lors du parsing.

La modélisation des orientations à l'aide d'un *sealed trait* garantit que tous les cas possibles sont connus à la compilation. Cela permet un traitement exhaustif et sécurisé, le compilateur signalant toute situation non couverte. Par rapport à une succession de conditions **if/else**, le pattern matching offre une meilleure lisibilité, une structure plus expressive et une robustesse accrue. Ce choix contribue ainsi à la clarté et à la fiabilité de l'implémentation.

3. Utilisation des collections fonctionnelles

Le projet exploite plusieurs méthodes issues des collections fonctionnelles Scala, notamment **foldLeft**, **map**, **grouped** et **zipWithIndex**. Le choix le plus structurant est l'utilisation de **foldLeft** dans le simulateur pour appliquer les instructions de déplacement.

Cette méthode permet d'appliquer les instructions de manière séquentielle tout en conservant un état accumulé. Chaque instruction transforme l'état courant de la tondeuse et produit un nouvel état qui devient l'entrée de l'itération suivante. Ce mécanisme correspond naturellement au problème posé, tout en respectant l'immutabilité des objets.

Les autres méthodes (**map**, **grouped**, **zipWithIndex**) sont utilisées pour transformer les données du fichier, regrouper les informations par paires et numéroté les tondeuses. Leur utilisation rend le code plus concis, plus déclaratif et plus expressif qu'une approche impérative classique.

4. Gestion des erreurs

Une attention particulière a été portée à la gestion des erreurs afin d'éviter les comportements imprévisibles. La lecture du fichier est encapsulée dans un **Try**, permettant de distinguer explicitement les cas de succès et d'échec sans provoquer d'arrêt brutal du programme. Cette approche offre un traitement contrôlé des exceptions et permet à l'objet `Main` de gérer proprement les erreurs via un pattern matching.

L'utilisation de Option lors de la conversion d'une chaîne de caractères en orientation permet également de gérer les valeurs invalides de manière explicite. En cas de format incorrect ou d'instruction invalide, des exceptions spécifiques sont levées afin de signaler clairement le problème.

Ces mécanismes sécurisent la lecture du fichier, renforcent la robustesse globale de l'application et garantissent un comportement prévisible en cas d'erreur.

IV. Tests et validation

Afin de garantir la fiabilité de la solution, plusieurs scénarios de test ont été réalisés. Le premier test correspond au cas nominal fourni dans l'énoncé, avec une grille de dimension 5x5 et deux tondeuses recevant des séquences d'instructions distinctes.

Par exemple :

Entrée :

```
5 5
1 2 N
GAGAGAGAA
3 3 E
AADAADADDA
```

Sortie obtenue :

```
Tondeuse 1 : 1 3 N
Tondeuse 2 : 5 1 E

Process finished with exit code 0
```

Le programme produit correctement les positions finales attendues, confirmant la validité de la logique algorithmique et du traitement séquentiel des instructions. Des tests complémentaires ont également été effectués pour vérifier la robustesse du système. Un scénario de dépassement des limites a permis de s'assurer qu'une tondeuse ne peut pas sortir de la grille : si un déplacement potentiel excède les bornes définies, la tondeuse reste immobile, conformément

aux spécifications. Un test avec une instruction invalide a confirmé que le programme lève une exception explicite, garantissant une gestion contrôlée des erreurs. Enfin, un test avec un fichier vide ou mal formaté permet de vérifier que la lecture du fichier est correctement encapsulée dans un Try, évitant tout arrêt brutal du programme.

Ces différents cas de validation montrent que la solution est non seulement fonctionnelle dans le cas standard, mais également robuste face aux situations imprévues. L'architecture adoptée facilite par ailleurs l'ajout futur de tests unitaires si nécessaire.

CONCLUSION

Ce projet a permis de concevoir et d'implémenter une solution complète de simulation en Scala, combinant rigueur algorithmique et structuration logicielle. Au-delà du simple calcul des positions finales des tondeuses, l'objectif était de proposer une architecture claire et cohérente, respectant les principes de la programmation orientée objet et les bonnes pratiques du langage.

La séparation des responsabilités, l'organisation en packages distincts (**model**, **service**, **Main**), l'adoption de l'immuabilité et l'utilisation du pattern matching ont contribué à produire un code lisible, maintenable et robuste. La gestion explicite des erreurs via **Try** et **Option** renforce la fiabilité de l'ensemble et garantit un comportement prévisible face aux situations invalides.

La solution répond pleinement aux exigences fonctionnelles du sujet tout en adoptant une approche moderne du développement en Scala, combinant programmation orientée objet et programmation fonctionnelle. Des extensions futures pourraient inclure l'ajout de tests unitaires automatisés, d'une interface graphique ou d'une version distribuée exploitant Spark, ouvrant ainsi des perspectives d'évolution plus avancées.